Metoda "dziel i zwyciężaj" konstrukcji algorytmów na przykładzie szybkiego mnożenia macierzy

Strategia bitewna: podziel - pokonaj - połącz
Oryginalne zadanie jest podzielone na mniejsze i potencjalnie łatwiejsze do rozwiązania podzadania, których wyniki otrzymane rekurencyjnie są scalane w wynik końcowy. Zadania o rozmiarze poniżej ustalonej stałej są rozwiązywane zwykłą metodą. Podział podzadań może wymagać dodatkowych obliczeń.

Twierdzenie 1 (O rekurencji uniwersalnej (Master Theorem))**.** *Załóżmy, że podział generuje $a \geq 1$ podzadań o rozmiarze $\lfloor \frac{n}{b} \rfloor$ lub $\lceil \frac{n}{b} \rceil$, gdzie $b \geq 1$ i złożoność algorytmu jest wyrażona równaniem*

$$T(n) = a \cdot T(n/b) + \Theta(n^p \log^q n), n > c,$$

$$T(n) = \Theta(1), n \leq c,$$

gdzie $p,\ q \geq 0$ i $\log_b a = d$.

1. *jeśli $d > p$, to $T(n) = \Theta(n^d)$,*
2. *jeśli $d = p$, to $T(n) = \Theta(n^d \log^{q+1} n)$,*
3. *jeśli $d < p$, to $T(n) = \Theta(n^p \log^q n)$.*

Algorytm Strassena

Wejście: $A, B \in \mathbb{R}^{n \times n},\ n = 2^k$
Wyjście: $C = A \cdot B$.

Traktujemy macierz jako 2×2, czyli podmacierze są rozmiaru $\frac{n}{2} \times \frac{n}{2}$.

$$A = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}$$

Standardowy algorytm wykonuje 8 mnożeń:

$$C_{i,j} = A_{i,i,} \cdot B_{i,j} + A_{i,j} \cdot B_{j,j},$$

$$T(n) = 8T\left(\frac{n}{2}\right) + b \cdot n^2 = \Theta(n^3)$$

Strassen zmniejszył liczbę mnożeń do 7. Jego metoda nie korzysta z przemienności mnożenia liczb, dzięki czemu może być wykorzystana do mnożenia podmacierzy.

1. Podziel macierze A, B na podmacierze $A_{1,1}, A_{1,2}, A_{2,1}, A_{2,2}, B_{1,1}, \ldots, B_{2,2}$.
2. Oblicz $C_{i,i} = B_{i,i} \cdot A_{i,i}$ rekurencyjnie.
3. Połącz wyniki $C_{i,i}$ wywołań za pomocą wzorów Strassena z rekurencyjnym mnożeniem.

Złożoność:

$$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2), n > 2,$$

$$T(n) = \Theta(n^{\lg 7}) \approx \Theta(n^{2.808})$$

Sortowanie przez scalanie i sortowanie szybkie.

Oba algorytmy są „Dziel i Zwyciężaj”.

Sortowanie przez scalanie

1. Posortuj osobno półówki tablicy.
2. Scal posortowane ciągi metodą dwóch wskaźników.

Złożoność:

$$T(n) = T(n/2) + T(n/2) = \Theta(n \log n)$$

Zalety i wady

- $\oplus$ złożoność pesymistyczna równa średniej
- $\oplus$ stabilność numeryczna
- $\ominus$ pamięć robocza wielkości $\Theta(n)$
- $\ominus$ złożoność czasowa przy posortowanych danych wejściowych również $\Theta(n)$

0.1 Sortowanie szybkie

Na każdym poziomie rekurencji chcemy uporządkować elementy tablicy A tak, żeby po lewej stronie pozycji p wszystkie wartości był mniejsze od tych po prawej.

Podział Hoare'a:

1. Ustal p.
2. Idąc wskaźnikiem i od lewej, znajdź $A[i] > p$.
3. Idąc wskaźnikiem j od prawej, znajdź $A[j] < p$.
4. Jeśli $i < j$, to zamień $A[i]$ z $A[j]$ i kontynuuj przesuwanie wskaźników.
5. Jeśli $i \geq j$, to zwróć p gdzieś w pobliżu i, j tak, żeby warunek był spełniony.

Piwot może być największym lub najmniejszym elementem – dobrze jest dodać wartowników.

Podział Lomuto:

1. Ustal p, ustaw wskaźnik i na początek tablicy.
2. Idąc wskaźnikiem j od początku do ostatniego elementu przed p.
3. Jeśli $A[j] \leq A[p]$, zwiększ i o 1, zamień $A[i]$ z $A[j]$.
4. Zamień $A[i+1]$ z $A[p]$.
5. Zwróć $p = i + 1$.

Wykonuje do trzech razy więcej zamian, szczególnie nieefektywna przy powtórzeniach wartości.

Można zastosować potrójny podział Lomuto, gdzie wartości równe piwotowi zostają zgrupowane na środku, a rekurencja jest wywoływana tylko na lewej i prawej podtablicy.

Dobre praktyki:

- Losowanie piwota – uniemożliwia zadanie złośliwych danych
- Wybór piwota jako mediany z $A[l], A[(l+r)/2], A[r]$ – poprawia działanie dla posortowanych ciągów, skonstruowanie złośliwego przypadku nadal jest możliwe
- Niesortowanie małych podzadań, a na końcu posortowanie całej tablicy przez wstawianie ($\Theta(n)$)

Złożoność

W przypadku oczekiwanym: $T(n) = 2T\left(\frac{n}{2}\right) + n + 1 = \Theta(n \log n)$

W przypadku pesymistycznym: $T(n) = T(n-1) + n + 1 = \Theta(n^2)$

Zalety i wady

- $\oplus$ efektywny w oczekiwanym przypadku
- $\oplus$ w miejscu - brak jawnej pamięci roboczej
- $\ominus$ niestabilność numeryczna
- $\ominus$ pamięć robocza pesymistycznie wielkości $\Theta(n)$ - niejawna, stos rekursji w pesymistycznym przypadku wymaga $\Theta(n)$ pamięci

Z ostatnim punktem można sobie radzić, sortując mniejszą podtablicę rekurencyjnie, a większą iteracyjnie.

Przegląd grafu w głąb, zastosowanie do znajdowania silnie spójnych składowych.

Algorytm DFS przechodzi po wierzchołkach, odwiedzając wszystkie na pewnej ścieżce i wycofując się dopiero, kiedy kolejne przejście jest niemożliwe. Koncepcyjnie przypomina metodę nawrotów (backtracking).

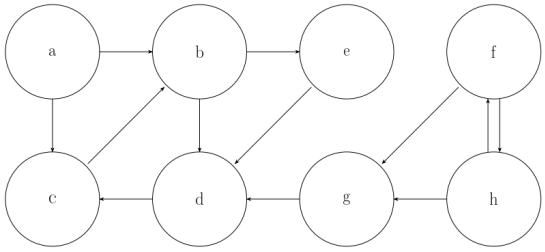
```
// G - graf, u - wierzchołek początkowy
// c[v] = WHITE - kolor v
// p[v] = NULL - poprzednik v w drzewie rozpinającym
// t_in[v] - czas wejścia do v (preorder)
// t_out[v] - czas wyjścia z v (postorder)
// time = 0 - zmienna globalna

DFS(G):
    for u in V(G):
        if c[u] == WHITE:
            DFS_visit(u)

DFS_visit(u):
    c[u] = GRAY
    t_in[u] = ++time
    for v in Adj[u]:
        if c[v] == WHITE:
            p[v] = u
            DFS_visit(v)
    c[u] = BLACK
    t_out[u] = ++time
```

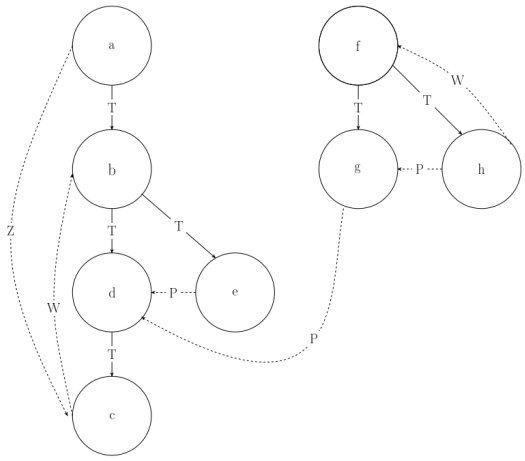
Krawędź (u, v) w drzewie DFS może być sklasyfikowana jako:

- drzewowa (T) – do następnika w drzewie
  `c(v) = WHITE`
- powrotna, wsteczna (W) – do przodka w drzewie
  `c(v) = GRAY`

- wzdłużna (Z) – do potomka w drzewie
  `c(v) = BLACK, t_in(u) < t_in(v)`
- poprzeczna (P) – do starszego poddrzewa
  `c(v) = BLACK, t_in(u) > t_in(v)`

W przypadku grafu nieskierowanego są tylko krawędzie drzewowe i powrotne.





Złożoność

Odwiedzane wierzchołki dostają kolor, więc funkcja `DFS_visit` jest wywołana dokładnie jeden raz dla każdego wierzchołka. Pętla `for` przeglądająca sąsiadów ma koszt $\Theta(\mathrm{Adj}(u))$, więc suma iteracji tej pętli dla wszystkich wierzchołków jest równa $\Theta(m) \longrightarrow$ całkowita złożoność: $\Theta(n+m)$.

Silnie spójne składowe

W grafie skierowanym $G = (V, E)$ silnie spójna składowa to maksymalny w sensie inkluzji podgraf indukowany, w którym dla każdej pary wierzchołków istnieje ścieżka z pierwszego do drugiego. Dowolne dwa wierzchołki są w obrębie jednej silnie spójnej składowej wtw leżą na cyklu.

1. Oblicz czasy postorder DFSem na grafie G.
2. Wyznacz graf odwrotny $G' = (V, E')$, gdzie $E' = \{(u, v) : (v, u) \in E\}$.
3. Wywołaj DFS na G' po malejących czasach postorder.

Każde drzewo lasu wygenerowanego w wywołaniu `DFS(GT)` jest oddzielną silnie spójną składową.

Dowód. Indukcja po kolejnych generowanych drzewach:
Przypadek bazowy – brak drzew.
Załóżmy, że drzewa $T_1, \ldots, T_{k-1}$ odpowiadają silnie spójnym składowym $C_1, \ldots, C_{k-1}$ w G. Z faktu, że dwa wierzchołki są w jednej silnie spójnej składowej wtw leżą na cyklu, wprost wynika, że spójne składowe w G oraz G' są takie same. Z korzenia T_k wychodzą więc białe ścieżki do wszystkich wierzchołków C_k, zatem $C_k \subseteq T_k$. Składowe $C_1, \ldots, C_{k-1}$ również mogą być osiągalne z korzenia T_k, ale w tym momencie przeglądu ścieżki prowadzące do nich są czarne, więc nie zostaną one uwzględnione w T_k.

W T_k nie wystąpi też żaden wierzchołek z $C_{k+1}, \ldots$ – dlaczego?
Jeśli $C_{k+i}\{C_1, \ldots, C_k\}$, to $\mathrm{t}[C_{k+i}] < \mathrm{t_out}[C_k]$. Gdyby istniała krawędź (u, v) z C_{k+i} do C_k, to $\mathrm{t}[C_{k+i}] > \mathrm{t}[C_k]$, czyli nie ma takiego połączenia. Nie może też być krawędzi z C_k do C_{k+i}. Wynika z tego, że $C_k = T_k$. $\square$

Złożoność algorytmu to $\Theta(n+m)$.

Przegląd grafu wszerz, podstawowe zastosowania.

Algorytm BFS odwiedza wierzchołki w kolejności niemalejącej odległości od wierzchołka początkowego.

```
// G - graf, u - wierzchołek początkowy
// c[v] = WHITE - kolor v
// d[v] = INF - odległość v od u
// p[v] = NULL - poprzednik v w drzewie rozpinającym
c[u] = GRAY, d[s] = 0, p[s] = NULL
Q.enqueue(s)
while !Q.empty() do
    u = Q.dequeue()
    for each v in Adj[u] do
        if c[v] == WHITE then
            c[v] = GRAY
            d[v] = d[u] + 1
            p[v] = u
            Q.enqueue(v)
    c[u] = BLACK
```

Dla każdego wierzchołka v, obliczona odległość d[v] rzeczywiście jest odległością od v do u, co można udowodnić indukcyjnie.

Złożoność

Ponieważ każdy wierzchołek v jest wstawiony do kolejki i usunięty z niej dokładnie jeden raz, to pętla `while` wykona się n razy. Również każda krawędź wychodząca z wierzchołka jest badana dokładnie raz, więc pętla `for each` w sumie wykona $2m$ iteracji (m dla grafu skierowanego) $\longrightarrow$ całkowita złożoność: $\Theta(n+m)$.

Zastosowania:

- Odległość wierzchołków od wierzchołka startowego
- Najkrótsza ścieżka łącząca dwa wierzchołki
- Spójna składowa wierzchołka
- Drzewo rozpinające – krawędzie $(p[u], u)$
- Dwudzielności grafu:

```
isBipartite(V, E, d):
    for each (u,v) in E do
        if (d[u] == d[v]) return false
    return true
```

Problem sortowania topologicznego, rozwiązanie za pomocą algorytmu DFS.

Dla DAGu $G = (V, E)$ zadanie polega na wyznaczeniu liniowego porządku $v_1, \ldots, v_n$ wierzchołków takiego, że, jeśli $(v_i, v_j) \in E$, to $i < j$. Jeśli graf zawiera cykle, taki porządek nie istnieje.

1. Wykonaj DFS i wyznacz kolejność postorder.
2. Zwróć odwróconą permutację czasów postorder.

Zamiast ostatniego kroku można trzymać stos roboczy i dodawać do niego wierzchołki w momencie kolorowania na czarno. Wynikiem jest porządek wierzchołków odpowiadający zdejmowaniu ich ze stosu.

Algorytm (w obu implementacjach) działa poprawnie.

Dowód. W momencie badania krawędzi (u, v) wierzchołek u jest szary, a wierzchołek v biały lub czarny, bo wpp krawędź (u, v) zamykałaby cykl. Jeśli jest biały, to:

$$\text{t_in}[u] < \text{t_in}[v] < \text{t_out}[v] < \text{t_out}[u]$$

Jeśli jest czarny, to:

$$\text{t_out}[v] < \text{time} < \text{t_out}[u],$$

ponieważ poddrzewo u nie jest jeszcze przeglądnięte w całości. Zatem, jeśli $(u, v) \in E$, to $\text{t_out}[v] < \text{t_out}[u]$. □

Zastosowanie

Planowanie kolejności i zrównoleglenia czynności, obliczanie najdłuższej ścieżki w grafie acyklicznym.

Kolejka priorytetowa, kopiec, zastosowanie do sortowania.

Kopiec (*heap*)

drzewo, w którym na dowolnej ścieżce w dół wartości maleją. Poziom liści jest wypełniony od lewej strony.

Kopiec można zaimplementować w tablicy A o rozmiarze $2^{\lfloor \lg n \rfloor}$:

- A[1] – korzeń
- A[2i] – lewy następnik A[i]
- A[2i+1] – prawy następnik A[i]
- A[i//2] – poprzednik A[i]

Jeśli wartość A[k] zwiększy się, trzeba uaktualnić wierzchołki na ścieżce od A[k] do korzenia.
Przesiewanie w górę (*sift-up*):

```
j = k, i = k//2
tmp = A[k]
while j > 1 and A[i] < tmp:
    A[j] = A[i]
    j = i, i = i//2
A[j] = tmp
```

Jeśli wartość A[k] zmniejszyła się, trzeba uaktualnić wierzchołki w poddrzewie A[k].
Przesiewanie w dół (*sift-down*):

```
tmp = A[k]
while k <= n//2:
    j = 2*k
    if j < n and A[j] < A[j+1]:
        j++
    if tmp >= A[j]:
        break
    A[k] = A[j]
    k = j
A[k] = tmp
```

Złożoność przesiewania: $\Theta(\lg n)$

Kolejka priorytetowa

może mieć pod spodem kopiec. Obsługuje trzy rodzaje operacji:

```
insert(x):
    A[++n] = x
    siftUp(A, n)
```

Złożoność: $\Theta(\lg n)$

```
max():
    return A[1]
```

Złożoność: $\Theta(1)$

```
deleteMax():
    A[1] = A[n]
    siftDown(A, 1, --n)
```

Złożoność: $\Theta(\lg n)$

Sortowanie metodą kopca (*heapsort*)

Żeby posortować ciąg n obiektów, wystarczy włożyć każdy z nich do kolejki, po czym n-krotnie wykonać operację deleteMax. Elementy opuszczają kolejkę w porządku malejącym. $\longrightarrow$ złożoność: $\Theta(n \lg n)$.

Ulepszeniem sortowania z wykorzystaniem kopca jest Heapsort (wersja Williamsa – zamiast wstawiania po jednym konstruujemy kopiec wywołując sift-down na wszystkich wierzchołkach od dołu (oprócz liści)
$\longrightarrow$ czas średni $\approx$ czas pesymistyczny.

Wersja Floyda ze zmodyfikowanym przesiewaniem zmniejsza średnią liczbę porównań z $2 \lg n$ do $\lg n + c$. Najpierw przesuwa w górę całą ścieżkę od korzenia do liścia (większy z następników jest przesuwany w miejsce rodzica). Wracając od liścia w górę, elementy mniejsze od przesiewanego są z powrotem przesuwane w dół.

Zalety i wady:

- $\oplus$ pesymistyczna złożoność $\Theta(n \lg n)$,
- $\oplus$ prostota implementacji,
- $\ominus$ dodatkowa tablica do przechowywania kopca.

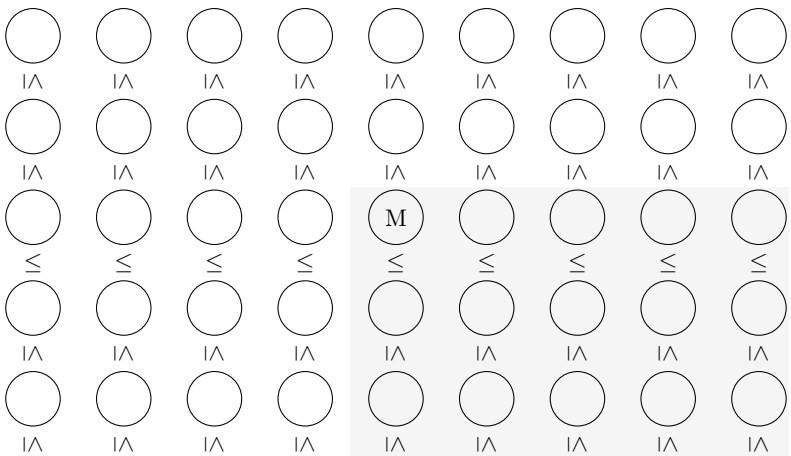
Znajdowanie k-tego elementu zbioru – algorytm „mediana median”, złożoność.

0.2 Algorytm mediana median

Algorytm wygląda następująco:

```
findK(S, k):
    if |S| < 50:
        sort(S)
        return S[k]
    S1, S2, ..., Sn/5 = split(S) // 5-elementowe podzbiory S
    sort(Si)
    Q = {median(S1), ..., median(Sn/5)}
    m = findK(Q, n/10)
    S1 = {x in S: x < M}
    S2 = {x in S: x = M}
    S3 = {x in S: x > M}
    if |S1| >= k:
        return select(S1, k)
    if |S1| + |S2| >= k:
        return M
    return findK(S3, k-|S1|-|S2|)
```

Złożoność



$S_1, S_3 \leq \frac{3}{4}n$

$$T(n) \leq T(n/5) + T(3/4n) + cn$$

$$T(n) \leq 20 \cdot cn = \Theta(n)$$

To wynika z twierdzenia:
Jeśli $c_i > 0$, $\sum_{i=1,\ldots,m} c_i < 1$, to rozwiązanie

$$T(n) = \sum_{i=1,\ldots,m} T(c_i \cdot n) + \Theta(n)$$

spełnia $T(n) = \Theta(n)$.

Dlaczego magiczne piątki?
Dla 3 równanie rekurencyjne jest postaci

$$T(n) = T(n/3) + T(2n/3) + O(n),$$

a $\frac{1}{3} + \frac{2}{3} \not< 1$, więc $T(n) = \Theta(n \log n)$. Dla 7, złożoność jest taka sama jak dla 5, ale posortowanie siódemek wymaga większej liczby porównań niż piątek.